AML SCREENING

Portal Project Documentation

Architecture, Menus, Database, Stored Procedures and Operational Flows

Implementation reference for Development, UAT and Production preparation

Document version: 1.0

Prepared: 1 September 2026

Application: AmlScreening.Portal (.NET 10 MVC)

Database: SQL Server - AMLScreening

Scope: Portal implementation; client API remains a separate future project

Document control

Item	Detail
Purpose	Describe the implemented AML Screening portal for developers, testers, administrators and compliance users.
System status	Portal workflow and database migration are implemented. The external client API is explicitly deferred.
Evidence basis	Current source code, applied SQL Server schema, live menu/permission catalog and passing integration tests.
Confidentiality	Internal project and controlled client/UAT use.

> **Current state:** The live database contains 19 dbo tables and 3 stored procedures. The portal uses EF Core and Dapper inline queries for its active paths; the procedures are documented as legacy/support objects and are not currently invoked by the C# repositories.

Contents

1. Project purpose and scope
2. Solution architecture and responsibilities
3. Authentication, authorization and tenant isolation
4. Menu-by-menu functional guide
5. End-to-end AML and compliance workflow
6. Risk scoring and decision logic
7. Database architecture and table dictionary
8. Stored procedure catalog and SQL definitions
9. Data access, audit, soft delete and concurrency
10. Permissions and access-control matrix
11. Configuration, migration, UAT and production guidance
12. Testing evidence, limitations and remaining work
13. Appendices: project structure, scripts and glossary

1. Project purpose and scope

The AML Screening Portal is a multi-client web application for managing screening lists, customer KYC profiles, compliance transactions, risk rules, screening evidence, automated decisions, alerts, cases, users, roles and audit history. It is designed as the operational portal that compliance staff use before a separate integration API is introduced.

The implemented prototype supports a practical transaction journey from customer creation to ALLOW, REVIEW or BLOCK. It is suitable for development and controlled UAT after environment configuration, but it is not yet a complete production sanctions platform because automated list ingestion, maker-checker approval, client API credentials and advanced transaction-monitoring models remain future work.

Implemented scope

- Cookie-based login with username/IP rate limiting and password hashing.
- Role permissions plus per-user grant or deny overrides.
- Database-driven sidebar menu definitions.
- Client-level query isolation and AML-record ownership controls.
- AML record add, edit, pagination and soft delete.
- Customer/KYC management and initial customer risk scoring.
- Transaction capture, screening, rule evaluation and persisted decisions.
- Compliance alerts, cases and audit records.
- Subscription plan management with soft delete and concurrency handling.
- Separate Development, UAT and Production configuration placeholders.
- SQL Server and in-memory integration tests.

Deferred scope

- External client API project, API credentials, OAuth2/mTLS and idempotency.
- Automated PEP, UN, EU, CIB and sanctions-list import/version pipelines.
- Country-risk masters, customer baselines, structuring detection and network analytics.
- Maker-checker approval workflow for rule and watchlist changes.
- Evidence attachments, regulatory STR/SAR submission and production case notes timeline.
- Distributed cache, message bus, outbox workers and search engine integration at scale.

2. Solution architecture and responsibilities

```
Browser / Razor Views
        -> MVC Controllers and authorization policies
        -> Application interfaces and authentication models
        -> Infrastructure repositories and workflow services
        -> EF Core / Dapper / SQL Server
        -> Audit evidence and client-scoped records
```

Layer/project	Responsibility
AmlScreening.Portal	MVC controllers, Razor views, authentication cookie, rate limiter, authorization handlers and current-user context.
AmlScreening.Application	Interfaces, permission constants, login service, DTOs and workflow result models.
AmlScreening.Domain	Business entities for users, clients, AML records, customers, transactions, rules, decisions, alerts and cases.
AmlScreening.Infrastructure	EF Core DbContext, Dapper connection factory, repositories, menu service, audit service and AML workflow engine.
Database	Versioned SQL scripts for isolation, permissions, menus, compliance rules, workflow tables and deployment preparation.
IntegrationTests	Cross-client checks, schema checks and repository behavior tests.
DatabaseMigrator	Small command-line runner for controlled execution of SQL migration scripts.

Design principles used

- Single responsibility: controllers coordinate HTTP behavior; repositories/services perform data and business work.
- Dependency inversion: controllers depend on application interfaces rather than concrete SQL implementations.
- Encapsulation: current user, client and permissions are exposed through ICurrentUserContext.
- Open/closed direction: new menu items, permissions and rules can be added without rewriting the layout.
- Defense in depth: menu filtering improves usability while controller policies and query scoping enforce security.

3. Authentication, authorization and tenant isolation

Login flow

1. The user submits a username, password and optional remember-me selection.
2. The login rate limiter partitions attempts by normalized username and remote IP address.

3. UserRepository loads the active user and active role from Users, UserRoles and Roles.
4. PasswordService verifies the stored password hash in application code.
5. Role permissions are combined with UserPermissions overrides. IsGranted allows an explicit grant or deny.
6. The portal creates claims for user ID, ClientId, username, full name, role and permissions.
7. The secure authentication cookie is issued and the user is redirected to Dashboard.
8. A login audit event is written; an audit failure is logged without preventing a valid login.

> **Rate limit:** The current fixed window permits five login POST attempts per username and IP combination per minute. Browser tabs share the same cookie, so opening more tabs does not create separate login sessions.

Permission resolution

```
Role permission exists and no user override -> granted
Role permission exists and user override IsGranted = 0 -> denied
Role permission absent and user override IsGranted = 1 -> granted
Neither role permission nor user grant exists -> denied
```

Menu visibility is based on MenuItems.PermissionCode. Actual access is separately enforced by [Authorize(Policy = ...)] on controllers and actions. A hidden menu is therefore not treated as the security boundary.

Client and owner scope

Control	Use
ClientId	Separates organizations. Client-scoped data queries are limited to the logged-in user's ClientId unless the platform-wide permission applies.
OwnerUserId	Used by AmlRecords to restrict ordinary users to records they created.
AMLRecords.ViewClient	Allows a user to view all AML records belonging to the current client, including records created by an administrator.
AMLRecords.ViewAll	Platform-level access across clients. This permission also bypasses EF Core client filters and must be assigned only to trusted platform administrators.

> **Current limitation:** Customers, transactions, alerts and cases are client-wide rather than owner-user-specific. If each client employee must see only their own customer workflow records, add OwnerUserId and resource authorization to those tables and services.

4. Menu-by-menu functional guide

Menu definitions are loaded from dbo.MenuItems. ParentMenuId creates the Compliance submenu. PermissionCode controls visibility, while ControllerName and ActionName identify the MVC destination.

Dashboard

Route: /Dashboard/Index

Required permission: Authenticated user

Provides a summary landing page after login. It combines AML record metrics and compliance workflow counts.

Main operations:

- Show total and active AML records.
- Show transaction count and decisions requiring review or block.
- Show open alert and open case counts.
- Display identity and role information from authentication claims.

Primary tables: AmlRecords, AmlTransactions, ComplianceAlerts, ComplianceCases

> **Important:** Dashboard values are now loaded from repositories/services rather than remaining fully hardcoded. Trend visuals may still require historical aggregation for production accuracy.

AML Records

Route: /AmlRecords/Index

Required permission: AMLRecords.View

Maintains the AML screening list data used during transaction evaluation. Records can represent individuals or entities from PEP, UN, EU, CIB and other sources.

Main operations:

- View paginated records.
- Search by name, reference or source.
- Filter by record type and active status.
- Add records with AMLRecords.Create.
- Edit records with AMLRecords.Edit.
- Soft delete records with AMLRecords.Delete.

Primary tables: AmlRecords, AuditLogs

> **Important:** Ordinary users see their own records unless AMLRecords.ViewClient is granted. Client-wide master data should normally be managed by trusted users and shared within the same client.

Users

Route: /Users/Index

Required permission: Users.View

Creates and lists portal users for the permitted client scope. Clicking the user name opens individual access editing.

Main operations:

- List active and inactive users with pagination.
- Create a user and assign a role with Users.Create.
- Open individual access from the user name.
- Apply per-user permission grants or denies with Roles.Manage.

Primary tables: Users, UserRoles, Roles, UserPermissions, Permissions, AuditLogs

Roles & Permissions

Route: /Roles/Index

Required permission: Roles.Manage

Centralizes reusable access profiles such as Admin, AML User and Viewer.

Main operations:

- Create a role.
- View all available permissions.
- Assign or remove permissions from a role.
- Use role permissions as the default access baseline for users.

Primary tables: Roles, Permissions, RolePermissions, AuditLogs

> **Important:** Role permissions should hold common access. UserPermissions should be reserved for exceptions, which keeps access administration understandable.

Subscription

Route: /Subscription/Index

Required permission: Subscription.View

Maintains source subscription plans, durations and rates. This is the portal-side plan catalog for later API entitlement work.

Main operations:

- List and paginate plans.
- Create a plan with Subscription.Create.

- Edit a plan with Subscription.Edit.
- Soft delete a plan with Subscription.Delete.
- Use RowVersion to prevent silent concurrent overwrites.

Primary tables: AML_Subscription_Plan, AuditLogs

> **Important:** Subscription data is not yet connected to API credentials, usage quotas or transaction enforcement. Those belong to the future API project.

Compliance

Compliance is a parent menu without its own controller. Its children form the operational transaction-monitoring workflow.

Compliance > Customers

Route: /Customers/Index

Required permission: Customers.View

Maintains customer/KYC profiles and their initial risk indicators.

Main operations:

- View client customers.
- Create customer identity and KYC details.
- Calculate customer risk from PEP status and missing KYC factors.
- Edit with optimistic concurrency.
- Soft delete customers.

Primary tables: Customers, Clients, AuditLogs

Compliance > Transactions

Route: /Transactions/Index

Required permission: Transactions.View

Captures customer transactions and provides the Evaluate action that runs screening, rules, scoring and decisioning.

Main operations:

- List recent client transactions.
- Create a transaction linked to an existing customer.
- View transaction, customer, match, risk and decision details.
- Evaluate with Transactions.Evaluate.
- Reuse an existing persisted result when a transaction has already been evaluated.

Primary tables: AmlTransactions, Customers, ScreeningMatches, RiskAssessments, AmlDecisions, ComplianceAlerts, ComplianceCases, AuditLogs

> **Important:** The current list is capped at 500 recent records and should receive server-side pagination before large production volumes.

Compliance > Threshold Rules

Route: /ComplianceRules/Index

Required permission: ComplianceRules.View

Configures the transaction rules used by the AML workflow engine.

Main operations:

- Create amount-threshold or velocity rules.
- Set currency, threshold, lookback, transaction count and risk score.
- Set the resulting action: Allow, Review/Hold/Alert or Block.
- Control priority, rule version, approval status and effective dates.
- Soft delete obsolete rules.

Primary tables: ComplianceRules, Clients, AuditLogs

> **Important:** ApprovalStatus is currently editable directly; true maker-checker approval is recommended before production.

Compliance > Alerts

Route: /Alerts/Index

Required permission: Alerts.View

Shows generated compliance alerts that require analyst attention.

Main operations:

- View severity, reason, transaction and status.
- Assign an alert to a user.
- Move the alert through investigation statuses.
- Require a resolution when resolving or closing an alert with Alerts.Manage.

Primary tables: ComplianceAlerts, AmlTransactions, RiskAssessments, Users, AuditLogs

Compliance > Cases

Route: /Cases/Index

Required permission: Cases.View

Provides the formal investigation record created from an alert.

Main operations:

- View cases and detailed evidence.
- Assign an investigator.
- Record status, disposition and notes.
- Require a disposition for cleared, rejected, STR/SAR or closed outcomes with Cases.Manage.

Primary tables: ComplianceCases, ComplianceAlerts, AmlTransactions, Users, AuditLogs

Audit Logs

Route: /AuditLogs/Index

Required permission: AuditLogs.View

Provides traceability for important portal and workflow actions.

Main operations:

- View recent login, logout, create, update, delete, evaluate, alert and case actions.
- Restrict ordinary viewers to their client.
- Allow controlled platform-wide review for appropriately privileged administrators.

Primary tables: AuditLogs, Clients, Users

Removed or inactive menu concepts

Search Screening, Reports and Settings are not present in the active MenuItems catalog. Their permission names remain in dbo.Permissions for backward compatibility but should be retired or reintroduced only when working controllers and views exist.

5. End-to-end AML and compliance workflow

```
AML list data -> Customer/KYC -> Transaction -> Screening matches
              -> Compliance rules -> Risk assessment -> Decision
              -> ALLOW, or REVIEW/BLOCK -> Alert -> Case -> Resolution
```

1. **Prepare screening data.** An authorized user adds active PEP, UN, EU, CIB or other watchlist entries to AmlRecords.
2. **Create customer.** A customer reference and KYC details are saved. Customer risk is calculated immediately.
3. **Create transaction.** A transaction is linked to the customer, assigned a correlation ID and stored with status NEW.
4. **Screen parties.** Sender and receiver names are normalized and compared with active AmlRecords for the current client.

5. **Apply rules.** Active and effective AmountThreshold and Velocity rules are evaluated in priority order.
6. **Calculate risk.** Customer, transaction and screening scores are combined and persisted in RiskAssessments.
7. **Decide.** AmlDecisions records ALLOW, REVIEW or BLOCK together with reasons, model version, rule versions and list versions.
8. **Escalate.** REVIEW and BLOCK create a ComplianceAlert and ComplianceCase for analyst work.
9. **Investigate and close.** Compliance staff assign, investigate, document and resolve the alert/case.
10. **Audit.** Application actions are written to AuditLogs throughout the journey.

Example transaction

Stage	Example result
AML record	UN-000125, John Example, Individual, UN Sanctions
Customer	CUS-1001, John Example, PEP = Yes
Transaction	TXN-9001, NPR 1,200,000, Nepal to destination country
Screening	Name similarity 88%; potential match stored
Rule	AmountThreshold rule triggers Review and adds transaction risk
Result	Risk evidence persisted; REVIEW decision; alert and case opened

6. Risk scoring and decision logic

Prototype formula: Overall risk = Customer risk x 40% + Transaction risk x 40% + Screening risk x 20%.

Component	Current implementation
Customer risk	PEP adds 50; missing source of funds adds 15; missing occupation adds 10; final score is capped at 100.
Screening risk	Levenshtein name similarity. Sender match can gain 10 for matching date of birth and 5 for matching nationality. Below 75 is ignored; 75-94.99 is potential; 95-100 is confirmed.
Transaction risk	Triggered rules add configured RiskScore. NPR/amount-independent fallback: amount >= 1,000,000 sets at least 60; amount >= 500,000 sets at least 35.
Risk level	0-29 LOW; 30-59 MEDIUM; 60-79 HIGH; 80-100 CRITICAL.
Decision	Confirmed match or Block action -> BLOCK. Potential match, score >= 60, or Review/Hold/Alert action -> REVIEW. Otherwise -> ALLOW.

Production caution: This score is an explainable prototype, not a regulator-approved model. Before production it requires governance, validated thresholds, country/product/channel risk, false-positive testing, model approval and periodic review.

7. Database architecture and table dictionary

The applied SQL Server database contains 19 dbo tables. ClientId is the principal tenant boundary. Foreign keys preserve core relationships, filtered unique indexes protect active references, DeleteFg supports soft deletion and RowVersion supports optimistic concurrency where configured.

Core relationships

```
Clients 1 -> many Users, AmlRecords, Customers, Transactions, Rules, Alerts, Cases
Users many -> many Roles through UserRoles
Roles many -> many Permissions through RolePermissions
Users many -> many permission overrides through UserPermissions
Customers 1 -> many AmlTransactions
AmlTransactions 1 -> many ScreeningMatches
AmlTransactions 1 -> 1 RiskAssessment -> 1 AmlDecision
AmlTransactions -> ComplianceAlert -> ComplianceCase
MenuItems parent -> child MenuItems
```

Identity, access and navigation tables

Table	Purpose	Important columns and relationships
Clients	Tenant/organization master.	Id PK, ClientCode unique, ClientName, IsActive, CreatedDate. Parent of client-owned business data.

Table	Purpose	Important columns and relationships
Users	Portal login accounts.	UserId PK, ClientId FK, Username, FullName, PasswordHash, IsActive, LastLoginDate, audit dates.
Roles	Reusable access profiles.	RoleId PK, RoleName, IsActive, CreatedDate.
UserRoles	User-to-role bridge.	Composite relationship: UserId FK -> Users; RoleId FK -> Roles.
Permissions	Named application capabilities.	PermissionId PK, PermissionName unique, Description, IsActive.
RolePermissions	Role permission assignments.	Composite PK; RoleId FK -> Roles; PermissionId FK -> Permissions.
UserPermissions	Per-user grant/deny overrides.	Composite PK; UserId FK, PermissionId FK, IsGranted. Overrides the role result.
MenuItems	Database-driven sidebar catalog.	MenuId PK, ParentMenuId self-FK, MenuKey, DisplayName, controller/action, icon, PermissionCode, order, active flag.

AML and compliance workflow tables

Table	Purpose	Important columns and relationships
AmlRecords	PEP/sanctions/watchlist master data.	Id PK, ClientId FK, OwnerUserId FK, type/name/DOB/country/nationality/reference/source, active/audit/soft-delete/concurrency fields.
Customers	Customer/KYC profiles and customer risk.	Id PK, ClientId FK, CustomerReference unique per client, KYC fields, IsPep, RiskScore, RiskLevel, lifecycle fields.
AmlTransactions	Transactions submitted for evaluation.	Id PK, ClientId FK, CustomerId FK, reference, parties, amount/currency, countries/channel/date, correlation, decision and lifecycle fields.
ComplianceRules	Tenant rule configuration.	Id PK, ClientId FK, code/version/type/threshold/velocity/action/risk/priority/approval /effective dates, lifecycle fields.
ScreeningMatches	Evidence of watchlist comparisons.	Id PK, ClientId FK, TransactionId FK, AmlRecordId FK, subject/matched names, score/status/source/list version/fields.
RiskAssessments	Persisted component and overall risk.	Id PK, ClientId FK, TransactionId unique FK, component scores, overall score/level, model version, factors and calculation time.
AmlDecisions	Explainable ALLOW/REVIEW/BLOCK result.	Id PK, ClientId FK, TransactionId unique FK, RiskAssessmentId FK, correlation, reasons, rule/model/list versions.
ComplianceAlerts	Analyst alert work queue.	Id PK, ClientId FK, TransactionId/RiskAssessmentId FKs, severity/reason/status, AssignedUserId FK, resolution and lifecycle fields.
ComplianceCases	Formal investigation record.	Id PK, ClientId FK, AlertId/TransactionId FKs, status, AssignedUserId FK, disposition, notes, closed date and lifecycle fields.

Operational and commercial tables

Table	Purpose	Important columns and relationships
AuditLogs	Immutable-style application activity evidence.	AuditLogId PK, ClientId FK, UserId, Username, Action, EntityType, EntityId, Details, CreatedDate.
AML_Subscription_Plan	Subscription/source plan catalog.	PlanId PK, SourceCode/SourceName, DurationMonths, Rate, IsActive, audit/soft-delete/concurrency fields.

Data integrity controls

- Unique client references prevent duplicate active AML, customer, transaction, alert and case references within the same client.
- Foreign keys prevent orphaned client, customer, transaction, alert, user and permission relationships.
- Check constraints enforce positive transaction amounts, valid risk-score ranges and Y/N soft-delete flags.
- RowVersion detects concurrent edits for supported entities.
- EF Core query filters hide soft-deleted rows and enforce ClientId scope for mapped workflow entities.

8. Stored procedure catalog and SQL definitions

> **Verified live catalog:** Three stored procedures currently exist. Source-code search found no current C# call using CommandType.StoredProcedure or these procedure names. Active authentication and repositories use inline Dapper SQL or EF Core.

Procedure	Purpose	Current status
spa_AML_SubscriptionPlan_GetAll	Returns active subscription plans ordered by PlanId.	Legacy/support; not called by current SubscriptionRepository.
usp_User_GetForLogin	Returns an active user and role by username.	Legacy; missing ClientId and permission resolution required by current login result.
usp_User_UpdateLastLogin	Updates LastLoginDate for a user.	Legacy; current repository performs an inline SYSUTCDATETIME update.

dbo.spa_AML_SubscriptionPlan_GetAll

```
CREATE PROC dbo.spa_AML_SubscriptionPlan_GetAll
AS
BEGIN
    SET NOCOUNT ON;
    SELECT PlanId, SourceCode, SourceName, DurationMonths, Rate, IsActive
    FROM dbo.AML_Subscription_Plan
    WHERE IsActive = 1
    ORDER BY PlanId;
END;
```

Before reuse, add DeleteFg filtering and decide whether subscription plans are global or client-owned. Current repository behavior includes pagination, soft delete, auditing and concurrency that this procedure does not provide.

dbo.usp_User_GetForLogin

```
CREATE PROCEDURE dbo.usp_User_GetForLogin
    @Username VARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;
    SELECT u.UserId, u.Username, u.FullName, u.PasswordHash,
           u.IsActive, r.RoleId, r.RoleName
    FROM dbo.Users u
    INNER JOIN dbo.UserRoles ur ON u.UserId = ur.UserId
    INNER JOIN dbo.Roles r ON ur.RoleId = r.RoleId
    WHERE u.Username = @Username
      AND u.IsActive = 1
      AND r.IsActive = 1;
END;
```

> **Do not switch directly:** The current login requires ClientId and separately resolves role permissions plus UserPermissions.IsGranted overrides. This stored procedure must be updated and regression-tested before it can replace UserRepository.GetUserAsync.

dbo.usp_User_UpdateLastLogin

```
CREATE PROCEDURE dbo.usp_User_UpdateLastLogin
    @UserId BIGINT
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE dbo.Users
    SET LastLoginDate = GETDATE()
    WHERE UserId = @UserId;
END;
```

The active repository uses SYSUTCDATETIME for consistent audit time. If this procedure is retained, change GETDATE to SYSUTCDATETIME and add error/row-count handling.

Stored procedure recommendation

Do not move the explainable risk engine into a large stored procedure. Keep business decisions in AmlWorkflowService where they are versionable and testable. Stored procedures are most useful later for stable reporting queries, bulk watchlist import staging, audit export and operational maintenance.

- usp_AmlRecords_BulkImportFromStaging - validate and merge approved list versions.
- usp_ComplianceDashboard_GetSummary - aggregate high-volume dashboard metrics.
- usp_AuditLogs_GetPage - stable server-side audit paging for large history.
- usp_TransactionMonitoring_GetVelocityFacts - optional optimized feature query, while keeping final decision logic in C#.

9. Data access, audit, soft delete and concurrency

Component	Technology	Responsibility
AmlRecordRepository	EF Core	Paging, ownership scope, create/edit/soft delete, concurrency and transactional audit.
AmlWorkflowService	EF Core	Customer, transaction, screening, risk, decision, alert and case workflow.
ComplianceRuleRepository	EF Core	Client-scoped rule CRUD, soft delete and audit.
UserRepository	Dapper inline SQL	Login user, permission resolution, users, roles and per-user access.
RoleRepository	Dapper inline SQL	Role listing, creation and permission assignment.
SubscriptionRepository	Dapper inline SQL	Plan paging, CRUD, soft delete, row-version checks and audit.
MenuService	Dapper inline SQL	Loads active database menu items and filters them against user claims.
AuditService	Dapper inline SQL	Writes and reads client-scoped audit events.

Transaction boundaries

Create, update and delete operations that also write audit evidence use database transactions in the EF Core repositories/services. Either both the business change and audit event commit, or the operation rolls back. The login audit is deliberately best-effort so an audit storage problem does not reject valid credentials.

Soft delete

```
DeleteFg   = 'Y'
DeleteDate = current UTC date/time
DeleteBy   = logged-in username
IsActive   = false where the entity has an active flag
```

Soft deletion preserves evidence. EF Core query filters hide deleted mapped records. SQL/Dapper queries must explicitly include DeleteFg = 'N' where applicable.

Optimistic concurrency

RowVersion changes automatically whenever SQL Server updates a row. Edit forms return the original RowVersion. If another user has updated the record first, the update is rejected and the user receives a conflict message instead of silently overwriting newer data.

10. Permissions and access-control matrix

Menu	Visibility permission	Additional control
Dashboard	Authenticated	View summary
AML Records	AMLRecords.View	Create/Edit/Delete and ViewClient/ViewAll are separate
Users	Users.View	Create requires Users.Create; access editing requires Roles.Manage
Roles & Permissions	Roles.Manage	Create roles and assign permissions
Subscription	Subscription.View	Create/Edit/Delete are separate
Customers	Customers.View	Create/Edit/Delete are separate
Transactions	Transactions.View	Create and Evaluate are separate
Threshold Rules	ComplianceRules.View	Create/Edit/Delete are separate
Alerts	Alerts.View	Assignment/resolution requires Alerts.Manage
Cases	Cases.View	Investigation updates require Cases.Manage
Audit Logs	AuditLogs.View	Read-only audit access

Recommended role model

Role	Recommended access
Platform Admin	All permissions including AMLRecords.ViewAll; restrict to trusted platform operators.
Client Admin	Users, roles, client-wide AML records, customer/transaction/rule/alert/case management, but no cross-client permission.
AML Analyst	Client-wide AML view, customer/transaction evaluation, alerts and cases; no user/role administration.
Data Maintainer	AML list create/edit and optional delete; no case or user administration.
Viewer/Auditor	Read-only menus required for review; no create/edit/delete/manage permissions.

11. Configuration, migration, UAT and production guidance

Configuration

Setting	Use
ConnectionStrings:AMLDatabase	SQL Server connection. Use a secret store outside Development.
AllowedHosts	Use the real UAT or production hostname; never use wildcard in hosted environments.
DataProtection:ApplicationName	Keeps authentication-cookie key usage scoped to this application.
DataProtection:KeysPath	Persistent protected key directory required for hosted environments and multiple instances.
ASPNETCORE_ENVIRONMENT	Development, UAT or Production selects the appropriate configuration.

Database script order

1. Back up the target database and verify the backup.
2. Run ClientIsolation.sql when upgrading a database that lacks Clients/ClientId/OwnerUserId.
3. Run AuditLogs.sql.
4. Run RolesPermissions.sql.
5. Run MenuItems.sql.
6. Run PortalAmlWorkflow.sql to synchronize columns and create the workflow tables.
7. Run schema and cross-client integration tests.

8. Restart the application and sign in again so the authentication cookie contains current permissions.

UAT checklist

- Use a separate UAT database and UAT hostname.
- Seed separate test clients, users, roles and representative AML records.
- Prove users cannot access another ClientId by changing URL IDs.
- Verify user permission grants and explicit denies after logout/login.
- Test ALLOW, REVIEW and BLOCK transaction scenarios.
- Verify alert resolution and case disposition requirements.
- Verify soft-deleted rows disappear but audit history remains.
- Review logs without exposing connection strings, hashes or sensitive KYC values.

Production controls

- Use managed secrets, least-privilege database credentials and encrypted connections.
- Persist and back up data-protection keys; configure shared keys for multiple application instances.
- Centralize structured logs and health monitoring.
- Add database backup, restore testing, retention and disaster-recovery procedures.
- Review role assignments regularly and record privileged-access changes.
- Run security, load, regression and SQL integration tests in the release pipeline.

12. Testing evidence, limitations and remaining work

Check	Result	Evidence
Release build	Passed	0 errors and 0 warnings after workflow migration changes.
Integration tests	Passed	6 of 6 tests passed with SQL Server test connection enabled.
Login query	Passed	Admin user resolves with ClientId, active Admin role and 32 effective permissions.
Schema verification	Passed	Missing IsGranted/RowVersion columns added and seven workflow tables created.
Browser negative login	Passed	Incorrect test password returns Invalid username or password instead of a database-unavailable message.
Correct-password browser login	User verification	The user's real password was not requested or entered during automated verification.

Known limitations

- AML list data is currently client-owned. A centralized platform normally separates global authoritative lists from client-private lists.
- ListVersion is currently manual-v1; automated source/version ingestion is not implemented.
- Name matching is a basic normalized Levenshtein implementation and lacks aliases, scripts/transliteration, identifiers and entity ownership matching.
- Receiver screening does not use receiver date of birth, nationality or identifiers because those fields are not captured.
- Threshold-rule approval is not true maker-checker.
- New customer and workflow lists need server-side pagination for production scale.
- Subscription plans do not yet enforce API entitlements or quotas.
- Search Screening, Reports and Settings permissions are legacy entries without active menu/controller implementations.

Recommended next project: client API

Capability	Suggested endpoint	Use
Customer screening	POST /api/v1/customers/screen	Screen customer/KYC identity against active list versions.
Transaction evaluation	POST /api/v1/aml/evaluate	Idempotent evaluation returning ALLOW/REVIEW/BLOCK and evidence references.
Evaluation lookup	GET /api/v1/evaluations/{correlationId}	Retrieve the persisted explainable result.
Case lookup	GET /api/v1/cases/{caseReference}	Controlled client access to case status.
Health	GET /health	Readiness and dependency checks without exposing sensitive details.

The API should add client credentials, subscription/entitlement enforcement, idempotency keys, correlation middleware, API rate limits, request/decision logs, mTLS or OAuth2, health checks, telemetry and retry-safe processing. It should call shared application services rather than duplicate portal rules.

Implementation priority

1. Separate global watchlist master data from client-private data and implement versioned imports.
2. Add maker-checker approval for rules and watchlist publication.
3. Add pagination to customers, transactions, alerts and cases.
4. Add OwnerUserId/resource authorization where users must not share client-wide operational records.

5. Add country/product/channel risk masters and validated customer risk configuration.
6. Build the separate API project with credentials, subscriptions, idempotency and integration tests.
7. Add evidence attachments, case event history and STR/SAR workflow.
8. Complete performance, security, recovery and production-readiness testing.

Appendix A. Project structure

```
AmlScreening.Domain/
    Common/                 Base entity and client ownership contracts
    Entities/               User, Client, AML record, subscription and audit entities
    Entities/Compliance/    Rules, customers, transactions, matches, risks, decisions, alerts, cases

AmlScreening.Application/
    Interface/              Service and repository contracts
    Models/                 Login, paging, menu, audit and workflow result models
    Security/               Claim names and centralized permission constants
    Services/               Authentication orchestration

AmlScreening.Infrastructure/
    Data/                   EF Core DbContext and query filters
    Persistence/            SQL connection factory
    Repositories/           AML, users, roles, subscriptions and rules
    Services/               Menu, audit and AML workflow engine

AmlScreening/
    Controllers/            MVC endpoints and permission policies
    ViewModels/             Form and page models
    Views/                  Razor pages and shared forms/layout
    Services/               CurrentUserContext
    Security/               Resource authorization handler
    wwwroot/css/            Portal visual styles

Database/                   Migration and deployment scripts
AmlScreening.IntegrationTests/  Automated repository/schema/isolation tests
AmlScreening.DatabaseMigrator/  Controlled SQL script runner
```

Appendix B. Database scripts

Script	Purpose
AmlRecords.sql	Base AML record table definition and seed-friendly structure.
AmlRecords.SoftDelete.sql	Adds soft-delete support to existing AML records.
ClientIsolation.sql	Adds Clients, ClientId, OwnerUserId and client-scoped indexes/foreign keys.
AuditLogs.sql	Creates or synchronizes audit storage.
RolesPermissions.sql	Creates permission tables and seeds application permissions and baseline roles.
MenuItems.sql	Creates and seeds the active database-driven navigation tree.
ComplianceRules.sql	Creates the compliance rules table for a new database.

Script	Purpose
PortalAmlWorkflow.sql	Synchronizes required columns and creates Customers, Transactions, Matches, Risks, Decisions, Alerts and Cases.
PlatformHardening.sql	Broader hardening/reference script; review carefully before use on an existing database.

Appendix C. Status glossary

Term	Meaning
NEW	Transaction captured but not yet evaluated.
CLEAR	No qualifying AML screening match.
POTENTIAL_MATCH	Screening score between 75 and below 95.
CONFIRMED_MATCH	Screening score of 95 or higher in the prototype.
ALLOW	Transaction cleared by current screening/risk/rule logic.
REVIEW	Transaction held for analyst investigation.
BLOCK	Confirmed match or blocking rule action.
LOW / MEDIUM / HIGH / CRITICAL	Risk bands for scores 0-29 / 30-59 / 60-79 / 80-100.
DeleteFg = Y	Soft-deleted record retained for evidence but hidden from normal queries.
RowVersion	SQL Server concurrency token used to detect stale edits.

Conclusion

The portal now provides a coherent multi-client AML operations prototype: controlled login, database-driven permissions and menus, watchlist records, KYC customers, transactions, explainable risk and decisions, alerts, cases and audit history. The next major boundary is the client API and production-grade data governance. Future development should preserve the current client isolation, permission policies, audit evidence and explainability rather than bypassing them in new controllers, SQL procedures or API endpoints.